

Version 1:

I decided to use structs for each bin with the records in it. So I had a struct for Bin, which represents one grid cell within the overall bins. The Bin struct keeps track of min/max longitude and latitude, the bins coordinates, row and col, the number of records, and the records, which are stored as an array of Records. The Record struct keeps track of the record's id, longitude, latitude, date, temp, and the comment (which can be empty). I chose to do this since each bin and record has many attributes to keep track of, and we went over how to create structs during the lab. I wrote two helper functions, one to sort the records by date(ascending), and another to extract the contents of the optional comment in a record. In my getComment function, any string beyond the first numeric fields is stored in a buffer. If no comment exists, an empty string is returned.

In my main function, I used malloc to create a 2D array to hold all the bins. I used row and column, which are equal to the number of lat/long divisions. When opening each file containing the record, I created a new Record instance to keep track of that record's data. I use my helper function to decide what gets stored in the comment. Then, after checking if the record is within the binning region, and computing which bin it belongs to, I used realloc to place it into the Bins array. Before outputting, I use my helper function to sort the records in each array, first by date, if date is equal then by latitude, and if latitude is equal then by longitude (all ascending).

Version 2

The same Record and Bin structs were used as v1. I added a new command line argument, keyString. Now, before placing a record into its calculated bin, I used strstr to check if the keyword is contained in the comment. I chose to use strstr because it is a way to check if a substring exists in a string. It made it so i dont have to create a separate loop or function to check this.

Version 3:

I added 2 new command line arguments, startdate and enddate which define the valid time window, and only records within these dates will be used to compute the averages. Filtering the bins as I go makes it so I don't have to reinitialize and calculate the bins. If the bin has a valid record, the date is averaged rounded to the nearest whole number, and the temperature is averaged as a float and rounded to two decimal places. In this version of my

code, I purposefully skip over any comments, by using `fscanf(inputFile, "%*[^\\n]");` and `fgetc(inputFile);`.

Version 4:

I added two new command line arguments, `querylong` and `querylat` which specify the specific location we are trying to query. I calculate `longBinWidth` and `latBinHeight` using the max/mins and the number of divisions, and then use that to find `queryLongBin` and `queryLatBin`, which specifies the bin that the query location belongs to. Then, I read each record as I did before and check if its in the query bin boundaries, and only continue if it is within the boundaries. I did this to prevent unnecessary processing of bins that do not contain the query point.

Build script:

I used `-Wall` and `-std=c11` because that is what was specified in the last assignment, and I assume its the same for this one. I used `-Werror` to make sure there are absolutely no errors or even warnings in my program.

Script1:

I created a temporary file as specified in the directions. I used the `find` function to find any folder path with the contents of a `.Dat` file, and stored that into a list. I also kept track of how many files were found so that I could add that first integer to the input file that specifies how many record files there are. I added both the `filecount` and `filelist` to the temp file and ran it with the executable that [build.sh](#) creates and the other CLA

script2:

My script 2 is exactly the same as script 1. This is because the only difference is that in script2 we have a folder with subfolders with `.Dat` files, rather than just a folder with `.Dat` files (script1). Because the `find` function recursively searches the folderpath until it finds the `.Dat` files, it doesn't matter how many subfolders it has to traverse until it finds the files.